



Volume 4 - Diagnostics, build, tests et benchmark

Ce que font CMake, Make, CTest, les sanitizers, le port-check et le benchmark

Ce volume documente les commandes de maintenance et de preuve : compilation stricte, tests, diagnostics locaux, mesures de débit et instrumentations mémoire/-concurrency.

Table des matières

1	Configuration CMake	2
2	Compilation	2
3	make build, make release, make debug	2
4	Tests CTest	2
5	ASan + UBSan	3
6	ThreadSanitizer	3
7	Diagnostic de ports	3
8	Makefile : make port-check	3
9	Démonstrations Makefile	3
9.1	Message	3
9.2	Signal	4
10	Benchmark de décodage	4
11	Nettoyage	4
12	Lecture recommandée d'un résultat de validation	4

1 Configuration CMake

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DSTGS_WARNINGS_AS_ERRORS=ON
```

CMake configure C++20 sans extensions, recherche Threads, crée les bibliothèques `stgs_core` et `stgs_network`, puis les exécutables. Les warnings incluent `-Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion -Wformat=2; -Werror` les transforme en erreurs.

Source : `CMakeLists.txt`, lignes 1–104

2 Compilation

```
cmake --build build --parallel 8
```

Le générateur choisi par CMake compile les unités séparées puis lie :

- `stgs_ground_station`;
- `stgs_satellite_simulator`;
- `stgs_port_check`;
- `stgs_tests` si activé;
- `stgs_benchmark_decode` si activé.

Le paramètre `-parallel` ne modifie pas le comportement du programme : il parallélise seulement la compilation.

3 make build, make release, make debug

Le Makefile est une façade : CMake reste la source de vérité.

```
make build
make release
make debug
```

`make build` exécute d'abord `configure`, puis `cmake -build`. `release` et `debug` rappellent `build` en remplaçant `BUILD_TYPE`.

4 Tests CTest

```
make test
# equivalent fonctionnel apres build :
ctest --test-dir build --output-on-failure
```

Le binaire `stgs_tests` regroupe tests codec/CRC, application-signal, replay/santé et réseau loopback. CTest ne lance pas un test par fonction : il lance l'exécutable de test enregistré sous `stgs_unit_and_integration_test`.

Pourquoi les tests réseau loopback comptent ?

Ils exercent de vraies sockets locales et des scénarios de fragmentation/fermeture, là où un simple test du codec ne peut pas détecter les erreurs de cycle de vie de `poll()`, `accept()` ou `recv()`.

5 ASan + UBSan

```
make asan-test
```

Le Makefile crée `build-asan` en Debug avec `STGS_ENABLE_SANITIZERS=ON`. CMake ajoute `-fsanitize=address,undefined` et `-fno-omit-frame-pointer`. Puis CTest tourne avec :

```
ASAN_OPTIONS=detect_leaks=1
UBSAN_OPTIONS=print_stacktrace=1:halt_on_error=1
```

AddressSanitizer cible notamment accès hors limites, use-after-free et erreurs de durée de vie; UBSan cible des comportements indéfinis détectables à l'exécution. Ce n'est pas une preuve mathématique d'absence de bug.

6 ThreadSanitizer

```
make tsan-test
```

Une build séparée active `-fsanitize=thread`. Le projet interdit volontairement de combiner TSan avec ASan/UBSan dans la même configuration. TSan observe les accès concurrents exécutés par les tests et signale les data races détectées.

7 Diagnostic de ports

```
./build/stgs_port_check --host 127.0.0.1 --ports 9000:9010 --timeout-ms 200
```

Port check

```
main → parseArgs local → isLoopbackIpv4 → scanTcpPorts → probeTcpPort x N →
socket 0_NONBLOCK → connect → poll(POLLOUT) → getsockopt(SO_ERROR) → rapport
TerminalUi
```

Le parcours est séquentiel. Les états sont : OPEN, CLOSED, TIMEOUT ou ERROR. La latence couvre la tentative de connexion observée. La plage est limitée à 256 ports et la cible au loopback 127.0.0.0/8 (ou localhost).

Port OPEN n'est pas service STGS identifié

Le diagnostic ne réalise aucun handshake applicatif. Un autre serveur TCP local sera également classé OPEN.

8 Makefile : make port-check

```
make port-check HOST=127.0.0.1 PORTS=9000:9010
```

Après `make build`, la recette appelle exactement `stgs_port_check -host $(HOST) -ports $(PORTS)`.

9 Démonstrations Makefile

9.1 Message

```
make message-demo HOST=127.0.0.1 PORTS=9000:9010
```

Le Makefile envoie un message via TCP en utilisant `-discover-ports`, avec `-count 1`.

9.2 Signal

```
make signal-demo HOST=127.0.0.1 PORTS=9000:9010
```

La recette envoie 5 blocs de 256 samples à 2 fps : sinus 5 Hz, $f_s = 200$ Hz, amplitude 1.0, bruit gaussien $\sigma = 0.35$.

10 Benchmark de décodage

```
./build/stgs_benchmark_decode --frames 200000 --payload-size 256 --decoder-threads 4 --seed 123456789
```

Benchmark

parseArgs → pré-génération de N TelemetryFrame → encodeFrame pour chaque trame → BlockingQueue capacité 4096 → démarrage N workers → start steady_clock → push des trames encodées → workers: decodeFrame → join → frames_per_second = traitées / durée

Ce que mesure le benchmark

Il mesure volontairement le coût CPU du pipeline de file + `decodeFrame()`. Il exclut réseau, TCP framing, disque, terminal, santé et STGA. Le débit obtenu n'est donc pas le débit bout-en-bout de la station.

Bornes benchmark

Frames : 1..10 000 000. Payload : 0..4096. Workers : 1..256. La totalité du jeu est pré-générée avant le chronométrage ; le garde-fou sur N évite une allocation accidentelle démesurée.

11 Nettoyage

```
make clean
make distclean
```

`clean` demande au générateur CMake de nettoyer le build courant. `distclean` supprime les répertoires `build`, `build-*`, les sorties telemetry CSV/JSON et les fichiers `*.stgf`.

12 Lecture recommandée d'un résultat de validation

1. Build strict GCC/Clang : qualité de compilation et portabilité de base.
2. CTest : couverture fonctionnelle exécutée.
3. ASan/UBSan : erreurs mémoire/UB sur les chemins testés.
4. TSan : races sur les chemins concurrents testés.
5. Benchmark : tendance CPU locale, à interpréter selon payload et workers.

Aucun de ces outils pris isolément ne prouve l'absence totale de défaut ; leur intérêt vient de leur complémentarité.